PROYECTO INTEGRADOR · COMPUTACIÓN EN LA NUBE

Azure Speech Demo

Convertir texto en voz y voz en texto con Azure AI Speech, React y Express

Nombre del estudiante · Curso · Fecha

CONTEXTO

Problema y objetivo

Qué se quiso resolver y hasta dónde llega el proyecto

- La voz es una interfaz natural, pero implementarla desde cero exige modelos propios.
- Azure ofrece el servicio ya entrenado: solo hay que integrarlo bien.

- **Objetivo:** app web con texto a voz y voz a texto.
- **Límite:** sin usuarios, sin base de datos, sin despliegue.
- **Condición:** la clave no llega al navegador.

DISEÑO

Arquitectura

El SDK corre en dos sitios porque el micrófono solo existe en el navegador



Función	Dónde corre el SDK	Credencial
Texto a voz	Servidor	Clave de suscripción
Voz a texto	Navegador	Token temporal (~10 min)

PARTE 1

Configuración en Azure

Cuatro pasos en el portal

1. Crear un recurso **Speech** (Azure AI Services) en portal.azure.com
2. Grupo de recursos nuevo, región cercana (eastus), plan **Free F0**
3. Ir a **Claves y punto de conexión** y copiar KEY 1 y la región
4. Pegarlas en **server/.env** (ignorado por git)

```
SPEECH_KEY=<KEY 1>
SPEECH_REGION=eastus
```

Texto a voz en el servidor

Se valida la entrada y se devuelve un MP3 generado en memoria

```
const cfg = sdk.SpeechConfig.fromSubscription(KEY, REGION);
cfg.speechSynthesisVoiceName = voice;      // es-CO-SalomeNeural
cfg.speechSynthesisOutputFormat =
  sdk.SpeechSynthesisOutputFormat.Audio16Khz128KBitRateMonoMp3;

const synth = new sdk.SpeechSynthesizer(cfg, null);
synth.speakTextAsync(text, (result) => {
  synth.close();
  res.set("Content-Type", "audio/mpeg");
  res.send(Buffer.from(result.audioData));
});
```

Validaciones previas: texto no vacío · máximo 500 caracteres · voz dentro del catálogo.

Token temporal para el navegador

El servidor cambia la clave por un token que caduca en unos 10 minutos

```
// server/index.js
const r = await fetch(
  `https://${REGION}.api.cognitive.microsoft.com/sts/v1.0/issueToken`,
  { method: "POST",
    headers: { "Ocp-Apim-Subscription-Key": KEY } });
res.json({ token: await r.text(), region: REGION });
```

- Si el token se filtra, caduca solo; la clave permanece privada.
- Se pide uno nuevo cada vez que se inicia la grabación.

Voz a texto en el navegador

Reconocimiento continuo con texto parcial y texto final

```
const { token, region } = await (await fetch("/api/token")).json();
const cfg = sdk.SpeechConfig.fromAuthorizationToken(token, region);
cfg.speechRecognitionLanguage = lang;      // es-CO
const rec = new sdk.SpeechRecognizer(
  cfg, sdk.AudioConfig.fromDefaultMicrophoneInput());

rec.recognizing = (_s, e) => setPartial(e.result.text);
rec.recognized  = (_s, e) => addFinal(e.result.text);
rec.startContinuousRecognitionAsync();
```

Al detener o cambiar de pestaña se cierra el reconocedor para liberar el micrófono.

PARTE 3

Uso de la aplicación

Texto a voz

1. Escribir el texto
2. Elegir idioma y voz
3. Pulsar **Escuchar**

Voz a texto

1. Elegir el idioma
2. **Iniciar grabación** y permitir el micrófono
3. Hablar; **Detener** al terminar

Diez idiomas disponibles: español (CO, ES, MX, AR), inglés (US, GB), portugués, francés, alemán e italiano.

APRENDIZAJE

Habilidades que se pusieron en práctica

- **Cloud:** crear y configurar un recurso, claves, regiones, planes y costos
- **Backend:** API REST con Express, validación de entradas, manejo de errores
- **Frontend:** React con hooks, estado, efectos y limpieza de recursos
- **Seguridad:** secretos en variables de entorno, tokens de corta vida
- **Integración:** uso de un SDK en cliente y servidor
- **Calidad:** mensajes de error claros, accesibilidad básica, documentación

CIERRE

Conclusiones

- Un servicio en la nube se integra con poco código si se separan bien las responsabilidades.
- La clave se queda en el servidor; el navegador solo recibe tokens temporales.
- Siguientes pasos: traducción en vivo, guardar historial, desplegar en Azure App Service.

Repositorio: github.com/JeanPierreCarPe/azure-integrador